

THE OPEN DOOR: INSTANT CHATTING WITH WEBSOCKETS

Lesson Plan & Comprehensive Student Worksheet

Subject: Computer Science & Web Tech

Target Audience: Kids / Beginners

Module: Real-Time Communication (Session 1)

1. LESSON CONCEPT & STORYLINE OVERVIEW

Imagine you are at home eagerly waiting for live updates on an exciting **IPL cricket match**. You want to know if Virat Kohli hit a six or if your favorite team won! In traditional web setups, your phone has to repeatedly send a request to the server every few seconds—asking, *"Is there an update? Is there an update?"*. This method is known as **HTTP Polling**.

Polling works like constantly knocking on your friend's front door to ask if they are ready to play. It requires continuous effort, creates delay, consumes significant network data, and rapidly drains your device's battery!

The WebSockets Paradigm: "The Open Door"

WebSockets solve this problem by keeping a persistent, open connection between your browser (client) and the server. Instead of knocking repeatedly, it is like keeping your friend's door wide open or having an active two-way walkie-talkie! Information flows back and forth instantly in real-time as soon as an event happens without any waiting or battery-draining polling loops.

2. KEY CONCEPT BREAKDOWN & COMPARISON

Feature / Metric	Traditional Polling (HTTP)	WebSockets Protocol
Analogy	Knocking on a door repeatedly and asking "Any news?"	Keeping the door open continuously or using an active walkie-talkie.
Communication Type	One-way at a time (Half-Duplex request/response cycle).	Full Bi-Directional (Full-Duplex) simultaneous communication.
Connection Lifecycle	Opens and closes a new HTTP connection for every single check.	Establishes a single handshake connection that stays open continuously.
Latency & Speed	Slower; updates are delayed until the next request cycle.	Instant; updates are pushed in real-time (milliseconds).
Efficiency & Battery	High battery usage & heavy data overhead due to repeated HTTP headers.	Highly efficient; lightweight frames save device battery and bandwidth.
Real-World Applications	Static web pages, traditional news feeds, form submissions.	Live IPL cricket scores, WhatsApp instant chat, multiplayer online games.

3. HOW WEBSOCKETS WORK (3 SIMPLE STEPS)

Step 1: The Handshake (Opening the Door)

The client (browser) sends an initial HTTP request to the server saying, *"Can we upgrade to a WebSocket connection?"*. The server agrees, establishing a permanent link.

Step 2: Persistent Connection (Door Stays Open)

The connection remains active in the background. Neither side needs to reconnect or ask for permission to speak again.

Step 3: Bi-Directional Data Flow (Instant Talking)

Either the client or server can push data instantly at any time. When a run is scored in cricket or a chat message is typed, it arrives instantly!

4. INTERACTIVE CLASSROOM ACTIVITIES & DISCUSSION

Activity A: "Guess the Delay!" (Voting Game)

Think about how polling and WebSockets work. If 1,000 users are checking for a live goal in a soccer match, which method delivers the goal alert faster and doesn't crash the server? Raise your hand to vote and discuss with your partner!

Activity B: True or False Quiz

- "WebSockets keep the connection door open all the time." — **TRUE** ✓
- "Polling asks the server again and again for updates." — **TRUE** ✓
- "Polling uses less battery than WebSockets." — **FALSE** ✗ (Polling wastes battery by constantly asking!)

STUDENT WORKSHEET & PRACTICE QUESTIONS

Test Your Knowledge & Hands-On Coding — Session 1: Real-Time WebSockets

Student Name: _____

Date: _____

Score: _____ / 100

Instructions: Answer all questions below based on your understanding of WebSockets, traditional polling, bi-directional communication, and the step-by-step code exercises.

Question 1: Multiple Choice **CONCEPT CHECK**

Which real-world analogy best describes how WebSockets operate compared to HTTP polling?

- A) Sending a physical letter in the post box every day.
- B) Keeping a friend's door wide open or having an active walkie-talkie always on.
- C) Ringing a doorbell, waiting 10 minutes, and walking away without speaking.
- D) Reading a printed book from start to finish.

Question 2: Terminology Connection **BI-DIRECTIONAL**

What does "Bi-Directional Communication" mean in the context of WebSockets, and how does it differ from traditional request-response polling?

Write your answer here...

Question 3: Real-World Scenarios **APPLICATION**

List three everyday applications or internet services that rely on WebSockets for instant data delivery, and explain why real-time speed is essential for each.

- 1.
- 2.
- 3.

Question 4: True or False & Explanation **ANALYSIS**

State whether the following statement is TRUE or FALSE, and explain WHY:

"Traditional polling is more energy-efficient and saves phone battery better than WebSockets when receiving live cricket scores."

Answer: [TRUE / FALSE]

Explanation:

Question 5: Step-by-Step Connection Flow PROCESS

Briefly describe the 3 main steps involved in establishing and sending messages over a WebSocket connection from start to finish.

Step 1:

Step 2:

Step 3:

Question 6: Architectural Problem Solving CRITICAL THINKING

Suppose you are designing a multiplayer online racing game where 8 players need to see each other's precise car position every 50 milliseconds. Explain why traditional HTTP polling would fail in this scenario and why WebSockets are required.

Write your explanation here...

HANDS-ON LAB: STEP-BY-STEP WEBSOCKET CODES

Build Your First Real-Time Chat App with JavaScript & Node.js

Follow these step-by-step code blocks to build a simple WebSocket client and server! Read each step, analyze the code, and complete the practice questions below.

Step 1: Setting Up the WebSocket Server (Node.js)

This code runs on the server (like a cricket score center or chat server) and listens for incoming connections.

```
// Step 1: Import the WebSocket library and create a server on port 8080
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });
console.log("🚀 WebSocket Server is open on port 8080!");
// Listen for incoming client connections (The Handshake)
wss.on('connection', (ws) => {
  console.log("👋 A new user connected! Door is wide open.");
  // Send a welcoming message from server to client
  ws.send("Welcome to the Instant Chat Server!");
});
```

Step 2: Connecting from the Client Browser (JavaScript)

This JavaScript code runs in the client browser (your phone or computer) to connect to the server.

```
// Step 2: Establish the handshake connection from browser
const socket = new WebSocket('ws://localhost:8080');
// Event 1: Triggered when connection successfully opens
socket.addEventListener('open', (event) => {
  console.log("✅ Connected to the WebSocket server!");
  // Send a message immediately after joining
  socket.send("Hello Server! Virat Kohli just hit a 6! 🏏");
});
```

Step 3: Receiving Messages Real-Time (Bi-Directional Flow)

Both client and server listen for incoming messages and handle them instantly.

```
// Step 3A: Browser Client listening for server messages
socket.addEventListener('message', (event) => {
  console.log("📬 Live Message Received:", event.data);
});
// Step 3B: Server handling incoming messages & broadcasting back
wss.on('connection', (ws) => {
  ws.on('message', (message) => {
    console.log("Received message: ${message}");
    // Echo message back to client
    ws.send(`Server Echo: ${message}`);
  });
});
```

Question 7: Code Analysis & Completion CODING PRACTICE

Look at **Step 2** above. Fill in the missing function call to send a live score update message when a button is clicked:

```
const sendButton = document.querySelector('#sendBtn');

sendButton.addEventListener('click', () => {
  const liveScore = "SIX! Score is now 184/3!";

  // Fill in the blank below to send liveScore over the socket:
  _____ ;

});
```

Question 8: Event Handling CODE LOGIC

In **Step 3A**, what event name is passed to `addEventListener` to listen for incoming messages from the server? Write down the correct JavaScript event name.

Event Name: